

Two-Way Header & Doubly Linked Lists

Representation, Traversal, Insertion and Deletion

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Recall: Singly Linked List

In a singly linked list, every node contains:

<i>DATA</i> <i>NEXT</i>

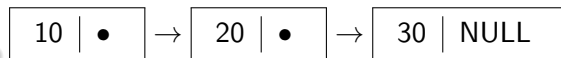
The NEXT field stores the address of the succeeding node.

But there is an important limitation:

Limitation

From a node, we can directly move to the next node, but not to the previous node.

START



Forward only

The Problem

Suppose PTR points to node 30.

$10 \rightarrow 20 \rightarrow \boxed{30} \rightarrow 40$

Moving forward is easy:

$PTR = PTR \rightarrow next$

But how do we directly return from 30 to 20?

The Problem

Suppose PTR points to node 30.

$10 \rightarrow 20 \rightarrow \boxed{30} \rightarrow 40$

Moving forward is easy:

$$PTR = PTR \rightarrow next$$

But how do we directly return from 30 to 20?

A singly linked node does not store that information.

From One-Way to Two-Way

Singly Linked Node



One link:



Doubly Linked Node



Two links:



Two-Way Linked List

What is a Doubly Linked List?

A **doubly linked list** or **two-way linked list**

contains two links in every node.

Each node consists of:

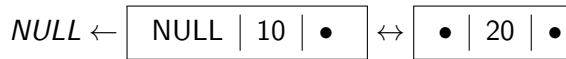
- 1 PREV
- 2 DATA
- 3 NEXT

PREV points to the preceding node.

NEXT points to the succeeding node.

Therefore traversal is possible in:

Both Directions



$\leftarrow$ Backward

Forward $\rightarrow$

Anatomy of a Doubly Linked Node

<i>PREV</i>		<i>DATA</i>		<i>NEXT</i>
-------------	--	-------------	--	-------------

PREV

Stores the address of the previous node.



DATA

Stores the actual information.

25

NEXT

Stores the address of the next node.



Boundary Rules

For the first node:

$$PREV = NULL$$

because no node exists before it.

For the last node:

$$NEXT = NULL$$

because no node exists after it.

START stores the address of the first node.

START



$$NULL \leftarrow [10] \leftrightarrow [20] \leftrightarrow [30] \rightarrow NULL$$

$$First.PREV = NULL$$

Memory Representation

Suppose:

$START = 100$

Address	PREV	DATA	NEXT
100	NULL	10	500
500	100	20	900
900	500	30	NULL

Logical representation:

$NULL \leftarrow 10 \leftrightarrow 20 \leftrightarrow 30 \rightarrow NULL$

Representing a Doubly Linked Node in C

```
struct Node
{
    struct Node *prev;
    int data;
    struct Node *next;
};
```

Corresponding logical node:

<i>prev</i>		<i>data</i>		<i>next</i>
-------------	--	-------------	--	-------------

This is a **self-referential structure** containing two node pointers.

Singly vs Doubly Linked List

Singly Linked List	Doubly Linked List
DATA + NEXT	PREV + DATA + NEXT
Forward traversal	Forward and backward traversal
Less memory per node	More memory per node
Previous node not directly available	Previous node directly available
Fewer pointer updates	More pointer updates
Simpler implementation	More flexible manipulation

Why Use a Doubly Linked List?

Advantages include:

- Bidirectional traversal.
- Easy access to previous node.
- Easier deletion when node address is known.
- Convenient insertion before a node.
- Useful for navigation systems.

10 ↔ 20 ↔ 30 ↔ 40

From 30:

30.*next* → 40

and:

30.*prev* → 20

Applications of Doubly Linked Lists

Common applications:

- Browser Back/Forward navigation
- Undo/Redo systems
- Music playlists
- Image viewers
- LRU cache
- Deque implementation
- Navigation histories

Browser Example

$Page_1 \leftrightarrow Page_2 \leftrightarrow Page_3$

Back

Forward

Forward Traversal

Forward traversal is similar to a singly linked list.

Start:

$$PTR = START$$

START

Process DATA and move:

$$PTR = PTR \rightarrow next$$



Stop when:

$$PTR = NULL$$

10 → 20 → 30 → 40 → *NULL*

Time complexity:

$$O(n)$$

PTR →

Forward Traversal in C

```
void displayForward(struct Node *start)
{
    struct Node *ptr = start;

    while (ptr != NULL)
    {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
}
```

For:

10 ↔ 20 ↔ 30

output is:

Backward Traversal

To traverse backward:
First reach the last node.
Then repeatedly use:

$$PTR = PTR \rightarrow prev$$

until:

$$PTR = NULL$$

For:

$$10 \leftrightarrow 20 \leftrightarrow 30$$

backward traversal produces:

30 20 10

Forward:

$$10 \rightarrow 20 \rightarrow 30$$

Backward:

$$30 \rightarrow 20 \rightarrow 10$$



Backward Traversal in C

```
void displayBackward(struct Node *start)
{
    struct Node *ptr = start;

    if (ptr == NULL)
        return;

    while (ptr->next != NULL)
        ptr = ptr->next;

    while (ptr != NULL)
    {
        printf("%d ", ptr->data);
        ptr = ptr->prev;
    }
}
```

Traversal: Dry Run

$NULL \leftarrow 10 \leftrightarrow 20 \leftrightarrow 30 \rightarrow NULL$

	Forward	Backward
Step 1	10	30
Step 2	20	20
Step 3	30	10

Same nodes, two traversal directions

Insertion Cases

We study four insertion cases:

- 1 Insert at the beginning.
- 2 Insert at the end.
- 3 Insert after a given node.
- 4 Insert before a given node.

Remember:

Two links may need updating

NEXT connection:



PREV connection:

Insertion at the Beginning

Suppose we insert 5 before 10.

Create NEW and perform:

$NEW \rightarrow prev = NULL$

$NEW \rightarrow next = START$

If list is non-empty:

$START \rightarrow prev = NEW$

Finally:

$START = NEW$

Before

$NULL \leftarrow 10 \leftrightarrow 20 \leftrightarrow 30$

After

$NULL \leftarrow \boxed{5} \leftrightarrow 10 \leftrightarrow 20 \leftrightarrow 30$

Insertion at Beginning: Pointer Changes

Before:



Create:



Perform:

$NEW \rightarrow next = START$

$START \rightarrow prev = NEW$

Insert at Beginning: C Code

```
struct Node *insertBeginning(  
    struct Node *start, int value)  
{  
    struct Node *newNode;  
  
    newNode = malloc(sizeof(struct Node));  
  
    if (newNode == NULL)  
        return start;  
  
    newNode->data = value;  
    newNode->prev = NULL;  
    newNode->next = start;  
  
    if (start != NULL)  
        start->prev = newNode;  
}
```

Insertion at the End

Suppose we insert 40.

First find the last node PTR.

Then:

$$PTR \rightarrow next = NEW$$

$$NEW \rightarrow prev = PTR$$

$$NEW \rightarrow next = NULL$$

Before

$$10 \leftrightarrow 20 \leftrightarrow 30 \rightarrow NULL$$

After

$$10 \leftrightarrow 20 \leftrightarrow 30 \leftrightarrow \boxed{40} \rightarrow NULL$$

Without a tail pointer, traversal is required.

$$\boxed{O(n)}$$

Insert at End: C Code

```
struct Node *insertEnd(  
    struct Node *start, int value)  
{  
    struct Node *newNode, *ptr;  
  
    newNode = malloc(sizeof(struct Node));  
  
    newNode->data = value;  
    newNode->next = NULL;  
  
    if (start == NULL)  
    {  
        newNode->prev = NULL;  
        return newNode;  
    }  
  
    ptr = start;
```

Insertion After a Given Node

Insert 25 after 20.

Suppose PTR points to 20.

Important links:

$$NEW \rightarrow prev = PTR$$

$$NEW \rightarrow next = PTR \rightarrow next$$

$$PTR \rightarrow next \rightarrow prev = NEW$$

$$PTR \rightarrow next = NEW$$

Before:

$$10 \leftrightarrow \boxed{20} \leftrightarrow 30$$

After:

$$10 \leftrightarrow 20 \leftrightarrow \boxed{25} \leftrightarrow 30$$

Boundary conditions must be handled if PTR is the last node.

Insertion After 20: Link Changes

Original:

10 ↔ 20 ↔ 30

New node:

25

Four relationships after insertion:

$20.next = 25$

$25.prev = 20$

Insert After a Node: C Code

```
void insertAfter(struct Node *ptr, int value)
{
    if (ptr == NULL)
        return;

    struct Node *newNode =
        malloc(sizeof(struct Node));

    newNode->data = value;

    newNode->prev = ptr;
    newNode->next = ptr->next;

    if (ptr->next != NULL)
        ptr->next->prev = newNode;

    ptr->next = newNode;
}
```

Insertion Before a Given Node

Insert 15 before 20.

If PTR points to 20:

$$NEW \rightarrow next = PTR$$

$$NEW \rightarrow prev = PTR \rightarrow prev$$

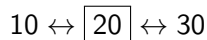
Then connect the previous node to NEW.

Finally:

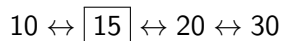
$$PTR \rightarrow prev = NEW$$

Unlike a singly linked list, PTR already tells us where its previous node is.

Before:



After:



Insert Before: C Code

```
1 struct Node *insertBefore(  
2     struct Node *start,  
3     struct Node *ptr,  
4     int value)  
5 {  
6     if (ptr == NULL)  
7         return start;  
8  
9     struct Node *newNode =  
10         malloc(sizeof(struct Node));  
11  
12     newNode->data = value;  
13     newNode->next = ptr;  
14     newNode->prev = ptr->prev;  
15  
16     if (ptr->prev != NULL)  
17         ptr->prev->next = newNode;  
18 }
```

The Golden Rule of DLL Insertion

When inserting between A and B:

$$A \leftrightarrow B$$

becomes:

$$A \leftrightarrow NEW \leftrightarrow B$$

Remember four relationships:

$$A \text{ next} \rightarrow NEW$$

Deletion Cases

The source considers four deletion cases:

- 1 Delete the first node.
- 2 Delete the last node.
- 3 Delete the node after a given node.
- 4 Delete the node before a given node.

The key advantage of PREV becomes clear during deletion.

$$PREV \leftarrow NODE \rightarrow NEXT$$

Delete the First Node

Suppose 10 is deleted.

Save:

$$PTR = START$$

Move START:

$$START = START \rightarrow next$$

Then:

$$START \rightarrow prev = NULL$$

Finally:

$$free(PTR)$$

Before:

$$NULL \leftarrow \boxed{10} \leftrightarrow 20 \leftrightarrow 30$$

After:

$$NULL \leftarrow 20 \leftrightarrow 30$$

Complexity:

Delete Beginning: C Code

```
struct Node *deleteBeginning(  
    struct Node *start)  
{  
    struct Node *temp;  
  
    if (start == NULL)  
        return NULL;  
  
    temp = start;  
    start = start->next;  
  
    if (start != NULL)  
        start->prev = NULL;  
  
    free(temp);  
  
    return start;  
}
```

Delete the Last Node

Suppose PTR points to the last node.

The second-last node is immediately available through:

$$PTR \rightarrow prev$$

Set:

$$PTR \rightarrow prev \rightarrow next = NULL$$

then:

$$free(PTR)$$

This demonstrates the usefulness of PREV.

Before:

$$10 \leftrightarrow 20 \leftrightarrow \boxed{30} \rightarrow NULL$$

After:

$$10 \leftrightarrow 20 \rightarrow NULL$$

Delete End: C Code

```
1 struct Node *deleteEnd(struct Node *start)
2 {
3     struct Node *ptr;
4
5     if (start == NULL)
6         return NULL;
7
8     ptr = start;
9
10    while (ptr->next != NULL)
11        ptr = ptr->next;
12
13    if (ptr->prev == NULL)
14    {
15        free(ptr);
16        return NULL;
17    }
18 }
```

Deleting a Middle Node

Suppose we delete 20:

$$10 \leftrightarrow \boxed{20} \leftrightarrow 30$$

PTR points to 20.

We must connect 10 directly with 30:

$$PTR \rightarrow prev \rightarrow next = PTR \rightarrow next$$

$$PTR \rightarrow next \rightarrow prev = PTR \rightarrow prev$$

Result:

Delete a Given Node

```
struct Node *deleteNode(  
    struct Node *start,  
    struct Node *ptr)  
{  
    if (ptr == NULL)  
        return start;  
  
    if (ptr->prev != NULL)  
        ptr->prev->next = ptr->next;  
    else  
        start = ptr->next;  
  
    if (ptr->next != NULL)  
        ptr->next->prev = ptr->prev;  
  
    free(ptr);  
}
```

Delete Node After a Given Node

Suppose PTR points to 20 and we want to delete its succeeding node 30.

Set:

$$TEMP = PTR \rightarrow next$$

Then:

$$PTR \rightarrow next = TEMP \rightarrow next$$

If another node exists:

$$TEMP \rightarrow next \rightarrow prev = PTR$$

Finally:

$$free(TEMP)$$

Before:

10 ↔ 20 ↔ 30 ↔ 40

After:

10 ↔ 20 ↔ 40

Delete Node Before a Given Node

Suppose PTR points to 30.
The node to delete is:

$$TEMP = PTR -> prev$$

Because TEMP also contains PREV, the node before TEMP can be accessed directly.
After reconnecting the neighbours:

$$free(TEMP)$$

Before:

10 ↔ 20 ↔ 30 ↔ 40

After:

10 ↔ 30 ↔ 40

A Major Advantage of Doubly Linked Lists

Suppose PTR already points to the node we want to delete.

In a singly linked list, we often need to find:

Previous Node

In a doubly linked list:

$PTR \rightarrow prev$

already gives us that node.

Therefore, given a node's address, local insertion/deletion can often be performed using a constant number of pointer updates.

What is a Header Node?

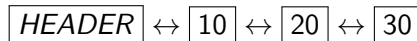
A **header node** is a special node placed before the actual data nodes.

It may:

- Store metadata.
- Store the number of nodes.
- Simplify boundary conditions.
- Act as a permanent starting node.

The header is generally not treated as an ordinary data element.

START



Actual data begins after HEADER.

Two-Way Header Linked List

A two-way header list combines:

Header Node

with:

PREV and NEXT links

A common implementation makes the header part of a circular structure.

This can remove many special cases involving:

- first node,
- last node,
- empty list.

HEADER ↔ 10 ↔ 20 ↔ 30

with the boundary links connected back to the header in a circular-header implementation.

Empty Two-Way Circular Header List

In an empty circular header list:

$HEADER \rightarrow next = HEADER$

and:

$HEADER \rightarrow prev = HEADER$

Conceptually:

$HEADER$

Non-Empty Two-Way Header List

Suppose the data values are:

10, 20, 30

Then conceptually:

HEADER ↔ 10 ↔ 20 ↔ 30 ↔ *HEADER*

Important relationships:

HEADER → *next* = *FIRST*

HEADER → *prev* = *LAST*

Why is a Header Node Useful?

Without Header

- Empty list requires $START = NULL$.
- First node is a special case.
- Last node may be a special case.
- More boundary checks.

With Circular Header

- Header always exists.
- Empty list has a valid structure.
- First/last nodes connect to header.
- Some algorithms become more uniform.

Header Node = Sentinel

Creating a Header Node in C

```
1 struct Node *createHeader()  
2 {  
3     struct Node *header;  
4  
5     header = malloc(sizeof(struct Node));  
6  
7     if (header == NULL)  
8         return NULL;  
9  
10    header->prev = header;  
11    header->next = header;  
12  
13    return header;  
14 }
```

Initially:

Insertion After Header

Initial list:

$HEADER \leftrightarrow 10 \leftrightarrow 20 \leftrightarrow HEADER$

Insert 5 immediately after HEADER.

Result:

$HEADER \leftrightarrow \boxed{5} \leftrightarrow 10 \leftrightarrow 20 \leftrightarrow HEADER$

The same general doubly-linked insertion idea applies:

$A \leftrightarrow NEW \leftrightarrow B$

Traversing a Circular Header List

```
void displayHeaderList(struct Node *header)
{
    struct Node *ptr = header->next;

    while (ptr != header)
    {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
}
```

Notice the stopping condition:

ptr != header

rather than:

Time Complexity

Operation	Complexity	Reason
Forward traversal	$O(n)$	Visit all nodes
Backward traversal	$O(n)$	Visit all nodes
Search	$O(n)$	Sequential search
Insert beginning	$O(1)$	Direct START update
Delete beginning	$O(1)$	Direct START update
Insert after known node	$O(1)$	Local pointer changes
Delete known node	$O(1)$	PREV and NEXT available
Insert end*	$O(n)$	Need to find last node

*Without maintaining a tail pointer or suitable header.

Complete Program: Structure and Display

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     struct Node *prev;
7     int data;
8     struct Node *next;
9 };
10
11 void display(struct Node *start)
12 {
13     struct Node *ptr = start;
14
15     printf("NULL <-> ");
16
17     while (ptr != NULL)
18     {
```

Complete Program: Insert Beginning

```
struct Node *insertBeginning(  
    struct Node *start, int value)  
{  
    struct Node *newNode =  
        malloc(sizeof(struct Node));  
  
    if (newNode == NULL)  
        return start;  
  
    newNode->data = value;  
    newNode->prev = NULL;  
    newNode->next = start;  
  
    if (start != NULL)  
        start->prev = newNode;  
  
    return newNode;  
}
```

Complete Program: Insert End

```
struct Node *insertEnd(  
    struct Node *start, int value)  
{  
    struct Node *newNode =  
        malloc(sizeof(struct Node));  
  
    newNode->data = value;  
    newNode->next = NULL;  
  
    if (start == NULL)  
    {  
        newNode->prev = NULL;  
        return newNode;  
    }  
  
    struct Node *ptr = start;
```

Complete Program: Delete Beginning

```
1 struct Node *deleteBeginning(  
2     struct Node *start)  
3 {  
4     if (start == NULL)  
5         return NULL;  
6  
7     struct Node *temp = start;  
8  
9     start = start->next;  
10  
11     if (start != NULL)  
12         start->prev = NULL;  
13  
14     free(temp);  
15  
16     return start;  
17 }
```

Complete Program: Main

```
int main()
{
    struct Node *start = NULL;

    start = insertEnd(start, 10);
    start = insertEnd(start, 20);
    start = insertEnd(start, 30);

    printf("Initial List:\n");
    display(start);

    start = insertBeginning(start, 5);

    printf("\nAfter inserting 5:\n");
    display(start);

    start = deleteBeginning(start);
```

Expected Output

Initial List:

```
NULL <-> 10 <-> 20 <-> 30 <-> NULL
```

After inserting 5:

```
NULL <-> 5 <-> 10 <-> 20 <-> 30 <-> NULL
```

After deleting first node:

```
NULL <-> 10 <-> 20 <-> 30 <-> NULL
```

Classroom Activity: Complete the Links

Given:

$10 \leftrightarrow 20 \leftrightarrow 30$

Insert 25 between 20 and 30.

What four links must be established?

Classroom Activity: Complete the Links

Given:

$10 \leftrightarrow 20 \leftrightarrow 30$

Insert 25 between 20 and 30.

What four links must be established?

$20.next = 25$

$25.prev = 20$

$25.next = 30$

$30.prev = 25$

Classroom Activity: Dry Run

Initial list:

10 ↔ 20 ↔ 30

Perform:

- ➊ Insert 5 at beginning.
- ➋ Insert 40 at end.
- ➌ Insert 25 after 20.
- ➍ Delete 10.
- ➎ Delete 40.

Classroom Activity: Dry Run

Initial list:

$10 \leftrightarrow 20 \leftrightarrow 30$

Perform:

- 1 Insert 5 at beginning.
- 2 Insert 40 at end.
- 3 Insert 25 after 20.
- 4 Delete 10.
- 5 Delete 40.

Final list:

$5 \leftrightarrow 20 \leftrightarrow 25 \leftrightarrow 30$

Find the Error

Suppose we insert NEW between PTR and PTR->next:

```
newNode->next = ptr->next;  
ptr->next = newNode;
```

What is missing?

Find the Error

Suppose we insert NEW between PTR and PTR->next:

```
newNode->next = ptr->next;  
ptr->next = newNode;
```

What is missing?

The PREV links!

We also need:

```
newNode->prev = ptr;  
  
if (newNode->next != NULL)  
    newNode->next->prev = newNode;
```

MCQ 1

How many link fields are present in a doubly linked node?

- ① 0
- ② 1
- ③ 2
- ④ 3

MCQ 1

How many link fields are present in a doubly linked node?

- ① 0
- ② 1
- ③ 2
- ④ 3

2

PREV and NEXT.

What is stored in PREV of the first node of a normal doubly linked list?

- ① START
- ② Address of last node
- ③ NULL
- ④ Address of next node

What is stored in PREV of the first node of a normal doubly linked list?

- ① START
- ② Address of last node
- ③ NULL
- ④ Address of next node

NULL

Which operation becomes easier because of the PREV pointer?

- ① Accessing the previous node
- ② Allocating an integer
- ③ Array indexing
- ④ Calculating array address

Which operation becomes easier because of the PREV pointer?

- ① Accessing the previous node
- ② Allocating an integer
- ③ Array indexing
- ④ Calculating array address

Accessing the previous node

MCQ 4

In an empty circular two-way header list:

- ① header is NULL
- ② header->next is NULL
- ③ header->next and header->prev point to header
- ④ header->prev is always NULL

In an empty circular two-way header list:

- ① header is NULL
- ② header->next is NULL
- ③ header->next and header->prev point to header
- ④ header->prev is always NULL

$$header \rightarrow next = header \rightarrow prev = header$$

Visual Recap

Singly

$10 \rightarrow 20 \rightarrow 30$

DATA|NEXT

One-way movement.

Doubly

$10 \leftrightarrow 20 \leftrightarrow 30$

PREV|DATA|NEXT

Two-way movement.

Summary

- A doubly linked list contains PREV, DATA and NEXT.
- PREV enables backward traversal.
- The first node has $PREV = NULL$.
- The last node has $NEXT = NULL$.
- Insertion requires maintaining both forward and backward links.
- Deletion is convenient when the node address is already known.
- Doubly linked lists require more memory than singly linked lists.
- Header nodes can simplify boundary conditions.
- A circular two-way header list uses the header as a sentinel.

Practice Questions

- 1 Write the C structure for a doubly linked node.
- 2 Write a program for forward and backward traversal.
- 3 Insert a node at the beginning and end.
- 4 Insert a node before a specified node.
- 5 Insert a node after a specified node.
- 6 Delete the first and last nodes.
- 7 Delete a given middle node.
- 8 Explain the role of a header node.
- 9 Compare singly and doubly linked lists.

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
- ② Reema Thareja, *Data Structures Using C*.
- ③ Horowitz, Sahni and Anderson-Freed, *Fundamentals of Data Structures in C*.

Your Feedback Matters

Please share your feedback about:

- PREV-DATA-NEXT visualization
- Forward/backward traversal
- Insertion operations
- Deletion operations
- Two-way header concept
- C programming examples



Scan to Submit Feedback

Thank You

Questions?

$NULL \leftarrow \boxed{10} \leftrightarrow \boxed{20} \leftrightarrow \boxed{30} \rightarrow NULL$

PREV + DATA + NEXT